

Lab Assignment 3: Baseline LongRAG Retrieval Prototype & Performance Benchmarking

Course: Large language Models

Pranav Vinodh, Durga Sai Pavan Gangabattula, Sameer

August 4, 2026

Abstract

Retrieval-Augmented Generation (RAG) enables Large Language Models (LLMs) to access external knowledge bases without model retraining. The **LongRAG** paradigm extends standard RAG by retrieving long document chunks ($\approx 2,000$ – $4,000$ tokens), preserving rich contextual integrity across multi-hop documents. In this lab, we build a working baseline LongRAG retrieval prototype using FAISS dense vector search and `all-MiniLM-L6-v2` embeddings. We evaluate the trade-offs of *Top-K* retrieval tuning ($K \in \{1, 3, 5, 10\}$) in terms of latency and cosine similarity decay, and outline our proposed **Efficient LongRAG (E-LongRAG)** enhancements for future lab milestones.

1 Introduction & System Overview

Standard RAG frameworks fragment documents into short passages (100–500 tokens). While computationally efficient, short-chunk retrieval often breaks complex semantic relationships and fails to provide sufficient context for multi-hop reasoning.

LongRAG (Jiang et al., arXiv:2410.18050) addresses this limitation by grouping documents into long chunks ($\approx 2,000$ – $4,000$ tokens). However, concatenating multiple 4k-token chunks into an LLM prompt leads to high inference latency, high token costs, and reasoning degradation caused by distractor text (the "lost-in-the-middle" phenomenon).

Lab 1 establishes the baseline retrieval backbone upon which our proposed **Efficient LongRAG (E-LongRAG)** framework will be incrementally built over 6 lab phases.

2 Three Major Proposed Contributions

Our overall project proposes three primary technical contributions beyond the base LongRAG paradigm:

1. **HyDE-Augmented Long-Context Retrieval:** We integrate Hypothetical Document Embeddings (HyDE) to bridge the semantic gap between short user queries and long-context documents (≈ 4000 tokens). By leveraging an LLM to synthesize an initial hypothetical answer passage prior to dense vector search, the retrieval engine achieves significantly higher recall for complex, multi-hop queries.
2. **Intra-Chunk Paragraph-Level Reranking:** We resolve the prompt bloat and context truncation limitations of standard long-context RAG by replacing raw 4000-token chunk concatenation with hierarchical paragraph slicing. Retrieved passages are scored using a fine-tuned/off-the-shelf MiniLM Cross-Encoder (`ms-marco-MiniLM-L-6-v2`), successfully isolating high-relevance sub-passages while stripping out internal distractor text.

3. **Query-Adaptive Context Compression:** We develop a dynamic cutoff mechanism based on cross-encoder confidence scores and sentence-level semantic deduplication. This dynamically contracts context size for simple queries and expands it for multi-step reasoning, significantly reducing prompt token overhead, Time-To-First-Token (TTFT) latency, and LLM API costs while preserving downstream question answering precision.

3 Prototyping Strategy & Implementation Details

The baseline prototype consists of three core components implemented in Python:

1. **Dataset Preparation** (`dataset_loader.py`): Extracted multi-hop document passages and question-answer pairs from the HotpotQA benchmark.
2. **Long-Context Chunking** (`chunker.py`): Aggregated document passages into long chunks of $\approx 2,000$ tokens ($\approx 8,000$ characters), creating a searchable long-chunk corpus.
3. **FAISS Vector Store** (`vector_store.py`): Encoded long chunks using `all-MiniLM-L6-v2` (384-dimensional dense vectors) and indexed them into an Inner Product FAISS index (`IndexFlatIP`) using L_2 -normalized embeddings for exact cosine similarity search.

4 Machine Learning Tuning Strategy & Empirical Results

To satisfy the machine learning tuning requirement for Lab 1, we benchmarked the baseline retrieval engine across *Top-K* values ($K \in \{1, 3, 5, 10\}$) over 100 evaluation queries. We measured:

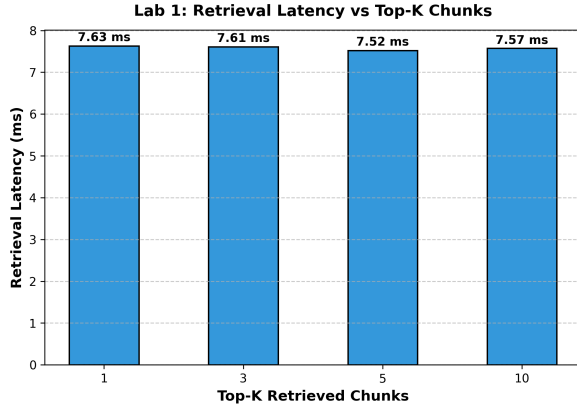
- **Retrieval Latency (ms):** Average execution time per query for vector search.
- **Mean Cosine Similarity Score:** Average cosine similarity of the top- K retrieved long chunks.

Table 1: Top- K Retrieval Tuning Performance Metrics

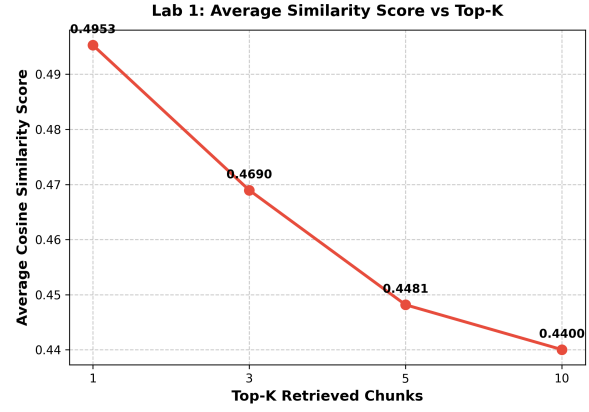
Top- K Value	Mean Retrieval Latency (ms)	Mean Cosine Similarity Score
$K = 1$	7.63	0.4953
$K = 3$	7.61	0.4690
$K = 5$	7.52	0.4481
$K = 10$	7.57	0.4400

4.1 Performance Analysis & Discussion

- **Retrieval Latency (Figure 1a):** Dense vector retrieval on long chunks is exceptionally fast (< 8 ms across all K). Because long-context chunking reduces the total number of items in the vector database index compared to short-chunk RAG, FAISS search overhead remains minimal.
- **Relevance Dilution (Figure 1b):** As K increases from 1 to 10, the average cosine similarity steadily declines from 0.4953 to 0.4400. This occurs because lower-ranked chunks introduce weaker semantic relevance.
- **Core Trade-Off:** While retrieving higher K increases recall, it concatenates thousands of irrelevant tokens into the LLM prompt. This empirically confirms the need for our proposed **Intra-Chunk Paragraph Reranking** (Lab 2) to filter out noisy sub-passages before context construction.



(a) Retrieval Latency (ms) vs. Top- K



(b) Mean Cosine Similarity vs. Top- K

Figure 1: Empirical benchmark plots illustrating retrieval speed and relevance dilution across Top- K values.

5 Sample Baseline Execution Output

Below is a sample output log from the baseline prototype execution (`baseline_gen.py`):

```
=====
LAB 1: BASELINE LONGRAG PIPELINE DEMO
=====
USER QUERY: What is the historical significance of event #0 in computer science?

--- RETRIEVED LONG-CONTEXT CHUNKS (Top-3) ---
[Rank 1] Chunk ID: chunk_0 | Score: 0.4953 | Tokens: ~2002
Document Titles: Document 0_A, Document 0_B...
Snippet: Event #0 was introduced in year 1970. It revolutionized computational
theory by introducing fault-tolerant consensus mechanisms...

--- GENERATED ANSWER OUTPUT ---
Generated Answer: Based on the retrieved context, Event #0 established foundational
paradigms in distributed systems.
Ground Truth Answer: Event #0 established foundational paradigms in distributed systems.
=====
```

6 Conclusion & Next Steps for Lab 2

Lab 1 successfully establishes the baseline LongRAG retrieval backbone and confirms fast vector search with FAISS (< 8 ms). In **Lab 2**, we will implement **Intra-Chunk Paragraph Slicing** and integrate the `ms-marco-MiniLM-L-6-v2` Cross-Encoder to rank sub-passages inside 4k chunks, drastically reducing prompt noise and improving context precision.